

Advance SQL Commands

This guide covers a set of powerful SQL techniques that go beyond the basics, helping you write more expressive, efficient, and flexible queries. What's covered:

- Timestamps & Extract
- Math Functions
- String Functions
- Sub-query
- Self-join

1. Timestamps & Extract

These commands report back time and date information. These are more useful when creating our own tables and databases, rather than when querying a database.

PostgreSQL can hold date and time information:

- **TIME** - Contains only time
- **DATE** - Contains only date
- **TIMESTAMP** - Contains date and time
- **TIMESTAMPTZ** - Contains date, time, and timezone.

Important Notes:

1. Careful considerations should be made when describing a table and database and choosing a time data type.
2. Depending on the situation you may or may not need the full level of **TIMESTAMPTZ**.
3. Remember, you can always remove historical information, but you can't add it!

Let's explore extracting information from a time based data type using:

- **EXTRACT():** Allows you to “extract” or obtain a sub-component of a date value.
 - YEAR
 - MONTH
 - DAY
 - WEEK
 - QUARTER

Example usage: `EXTRACT(YEAR FROM date_col)`

- **AGE():** Calculates and returns the current age given a timestamp.
 - Usage: `AGE(date_col)`
 - Returns: 13 years 1 mon 5 days 01:34:13.003423
- **TO_CHAR():** General function to convert data types to text.
 - Useful for timestamp formatting.
 - Example usage: `TO_CHAR(date_col, 'mm-dd-yyyy')`

2. SubQuery

A subquery allows you to construct complex queries, essentially performing a query on the results of another query. The syntax is straightforward and involves two **SELECT** statements.

Imagine a table consisting of student names and their test scores.

Standard Query

- **SELECT** student, grade
FROM test_scores

Standard Query to return average grade

- **SELECT AVG(grade)**
FROM test_scores

How can we get a list of students who scored better than the average grade?

We need to do 2 steps:

1. Get the average grade
2. Compare the table against the average grade.

This is where a subquery can help us get the result in a “single” query request.

```
SELECT student, grade
FROM test_scores
WHERE grade > (
    SELECT AVG(grade)
    FROM test_scores
)
```

The subquery is performed first since it is inside the parenthesis,

```
SELECT student, grade
FROM test_scores
WHERE grade > (SELECT AVG(grade)
FROM test_scores)
```

```
SELECT student, grade
FROM test_scores
WHERE grade > (70)
```



You can also use the IN operator in conjunction with a subquery to check against multiple results returned.

A subquery can also operate on a separate table.

```
SELECT student,grade
FROM test_scores
WHERE student IN
(SELECT student
FROM honor_roll_table)
```

```
SELECT student,grade
FROM test_scores
WHERE student IN
(('Zach' , 'Chris' , 'Karissa'))
```

The **EXISTS** operator is used to test for existence of rows in a subquery. Typically a subquery is passed in the **EXISTS()** function to check if any rows are returned with the subquery.

Syntax:

```
SELECT column_name FROM table_name
WHERE EXISTS (
    SELECT column_name FROM
    table_name WHERE condition
);
```

3. Self Join

A self-join is a query in which a table is joined to itself. They are useful for comparing values in a column of rows within the same table.

- The self join can be viewed as a join of two copies of the same table.
- The table is not actually copied, but sql performs the command as though it were.
- There is no special keyword for a self join, it's simply standard **JOIN** syntax with the same table in both parts.

***Important:** When using a self join it is necessary to use an alias for the table, otherwise the table names would be ambiguous.*

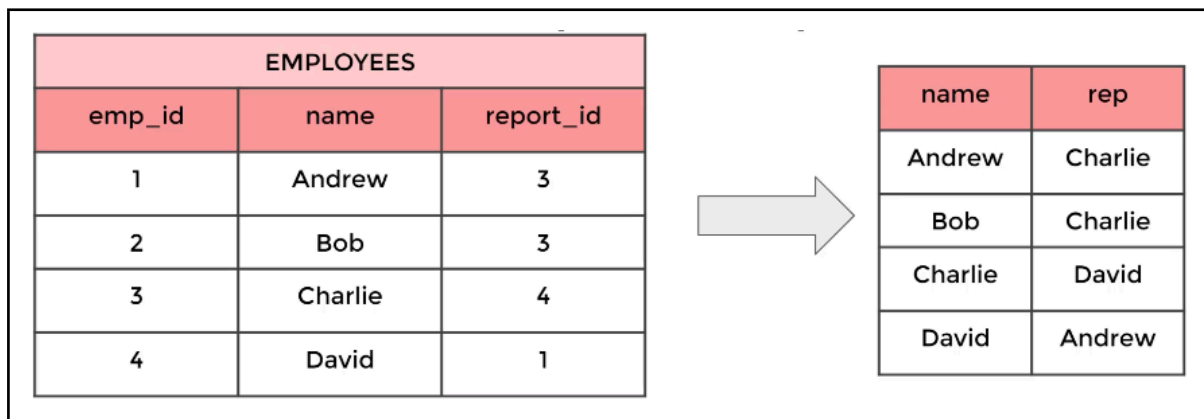
Syntax Example:

```
SELECT tableA.col, tableB.col  
FROM table AS tableA  
JOIN table AS tableB ON  
    tableA.some_col = tableB.other_col;
```

Let's explore a more realistic situation of when you would use this.

EMPLOYEES		
emp_id	name	report
1	Andrew	3
2	Bob	3
3	Charlie	4
4	David	1

We want results showing the employee name and their report's recipient name.



Query Solution:

```
SELECT emp.name , report.name  
FROM employees AS emp  
JOIN employees AS report ON  
    emp.emp_id = report.report_id;
```

name	rep
Andrew	Charlie
Bob	Charlie
Charlie	David
David	Andrew